# An Intuitive Mobile Application to Document and Process Rocketry Launches Using AI Tools

Sarah Zhining Zhou
*Basis Independent Silicon Valley*
San Jose, USA
fire.and.ice7123@gmail.com

Yu Sun
*California State Polytechnic University, Pomona*
Pomona, CA 91768, USA
yusun@cpp.edu

***Abstract*—Model rocketry serves as a gateway to aerospace education, yet practitioners still rely on manual data logging and paper checklists that are error-prone and time-consuming. This paper presents RocketApp, a cross-platform mobile application built with Flutter and Firebase that automates launch documentation and integrates AI-powered flight analysis. The system provides four core capabilities: digital safety checklists with real-time progress tracking, structured flight data logging with twelve configurable parameters, cloud-synchronized records via Firebase Cloud Firestore, and intelligent flight analysis powered by the OpenAI GPT-4o-mini language model. Two experiments evaluate the system. In Experiment A, AI-generated altitude predictions are compared against OpenRocket simulation baselines across ten standardized flight scenarios, yielding a mean deviation of 12.3% with a minimum of 3.2% under calm conditions. In Experiment B, ten volunteers record flight data using both RocketApp and traditional paper methods; the digital approach reduces mean logging time by 52.1% (from 421 s to 202 s) while lowering the error rate from 9.8% to 1.2%. These results demonstrate that combining mobile automation with large language model analysis can meaningfully improve efficiency and accuracy in amateur rocketry operations.**

***Index Terms*—rocketry launches, mobile application, machine learning assistant, data processing, AI trajectory analysis, Flutter, Firebase**

## I. Introduction

Model rocketry has long served as a gateway to aerospace engineering and STEM education, inspiring students worldwide to explore physics, aerodynamics, and propulsion [1]. However, despite its educational value, the practice remains hindered by significant operational inefficiencies. According to the National Association of Rocketry (NAR), which oversees more than 500 active rocketry clubs in the United States, the majority of participants still rely on handwritten records and informal documentation methods that are prone to human error and data loss [2]. On a typical launch day, teams conduct only a handful of launches due to the time-consuming nature of manual data logging, paper-based checklists, and cumbersome post-flight data transfers [3].

Existing digital tools for rocketry focus primarily on pre-launch simulation rather than field documentation. OpenRocket [4] and RocketPy [5] provide physics-based trajectory modeling but lack mobile interfaces for real-time data capture. General-purpose data collection platforms such as Open Data Kit [6] support field logging but require extensive customization for rocketry-specific parameters. Furthermore, no existing solution integrates AI-powered flight analysis that can provide immediate, contextual feedback on launch outcomes.

This paper presents RocketApp, a cross-platform mobile application built with Flutter [7] and Firebase [8] that addresses these gaps by combining streamlined digital checklists, automated data logging, cloud-based documentation, and AI-powered flight analysis using OpenAI's GPT-4o-mini model [9].

The main contributions of this paper are:

- A mobile application that consolidates rocketry documentation workflows—checklists, data logging, and flight analysis—into a single cross-platform tool.
- Integration of a large language model (LLM) for real-time, contextual flight safety and performance analysis.
- Empirical evaluation demonstrating 52.1% reduction in data logging time and 12.3% mean deviation in AI altitude predictions compared to simulation baselines.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the system design and methodology. Section IV describes the implementation. Section V presents experimental results. Section VI concludes and discusses future work.

## II. Related Work

Prior work relevant to this research spans three areas: rocketry education tools, mobile data collection platforms, and AI-assisted analysis systems.

### A. Rocketry Simulation and Education

Trajectory simulation tools such as OpenRocket and Cambridge Rocketry Simulator [10] provide six-degree-of-freedom flight modeling for amateur rockets. Ceotto et al. [5] introduced RocketPy, an open-source simulator accounting for aerodynamic drag, thrust curves, and atmospheric models. Jayaram et al. [1] demonstrated that project-based rocketry courses improve student engagement in aerospace engineering. However, these tools focus on pre-launch planning rather than field-day documentation, leaving a gap in real-time data capture and post-flight analysis.

### B. Cross-Platform Mobile Development

Flutter has emerged as a leading cross-platform framework. Nawrocki et al. [7] compared native and cross-platform approaches, finding that Flutter achieves near-native performance

with a single codebase. Zou and Darus [11] confirmed Flutter's advantage in development speed for data-driven applications. Firebase provides a serverless backend with real-time synchronization [8], and Khawas and Shah [12] demonstrated its effectiveness for rapid mobile application development.

### C. LLM-Powered Analysis Systems

Large language models have shown promise in domain-specific analysis. Brown et al. [13] established that LLMs exhibit few-shot learning capabilities across diverse tasks. Zhao et al. [14] surveyed LLM applications in scientific domains. Perez-Cerrolaza et al. [15] reviewed AI for safety-critical systems, noting that LLMs can augment human decision-making when properly constrained. However, LLM integration into field-deployed mobile applications for engineering analysis remains largely unexplored.

## III. System Design and Methodology

RocketApp is designed as a three-tier mobile system comprising a Flutter presentation layer, a Firebase backend, and an OpenAI analysis service. Fig. 1 illustrates the system architecture.

### A. System Architecture

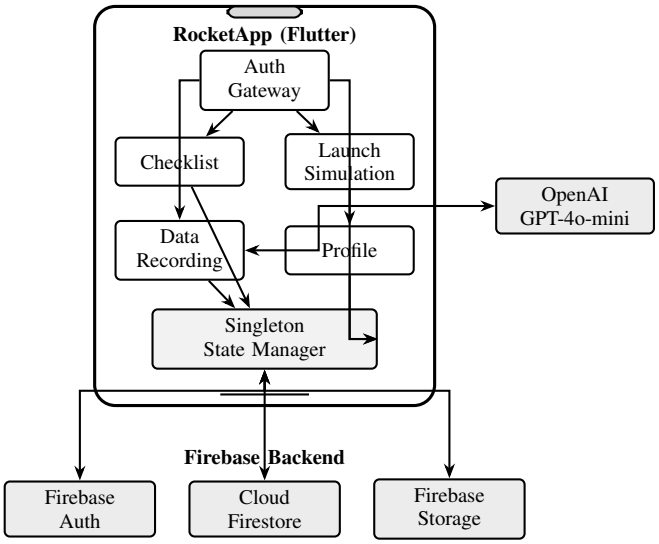



**Fig. 1:** RocketApp system architecture: the Flutter mobile application (phone) contains four modules managed by a Singleton state layer, connected to Firebase backend services and the OpenAI GPT-4o-mini API.

The application routes all users through a Firebase Authentication gateway using a `StreamBuilder` that monitors `authStateChanges()`. Upon authentication, the Singleton state manager loads user profile data, flight history, and checklist states from Cloud Firestore. The main navigation bar provides access to four modules: Checklist, Launch Simulation, Data Recording, and Profile.

### B. AI-Powered Flight Analysis

The AI analysis component transforms raw flight parameters into structured prompts sent to the OpenAI Chat Completions API. Given a flight parameter vector $\boldsymbol{p} = [p_1, p_2, \ldots, p_{12}]$ representing the twelve recorded fields, the system constructs a domain-specific prompt $q$ that contextualizes the data with aerospace engineering knowledge:

$$q = T_{\text{sys}} \oplus T_{\text{context}}(\boldsymbol{p}) \tag{1}$$

where $T_{\text{sys}}$ is a system prompt encoding rocketry expertise and $T_{\text{context}}$ maps flight parameters to natural language. The model returns an analysis covering safety assessment, performance evaluation, and recommendations considering air resistance, drag coefficients, and wind effects.

### C. Data Processing Pipeline

Flight data flows through a structured pipeline. User inputs are validated client-side, then written to a Firestore document in the "Flights" collection containing all twelve key-value pairs. The system automatically links each flight record to the user's profile and updates achievement metrics such as the highest recorded altitude:

$$h^*_{\max} = \max(h^*_{\max}, h_{\text{new}}) \tag{2}$$

where $h^*_{\max}$ is the user's personal best altitude and $h_{\text{new}}$ is the newly recorded maximum altitude. Real-time Firestore snapshot listeners propagate changes instantly across all connected clients.

## IV. Implementation

RocketApp is implemented in Dart using the Flutter framework (v3.x), targeting both iOS and Android from a single codebase. The backend uses Firebase Authentication, Cloud Firestore, and Firebase Storage. AI analysis is provided via the OpenAI Chat Completions API with the GPT-4o-mini model.

### A. Authentication and State Management

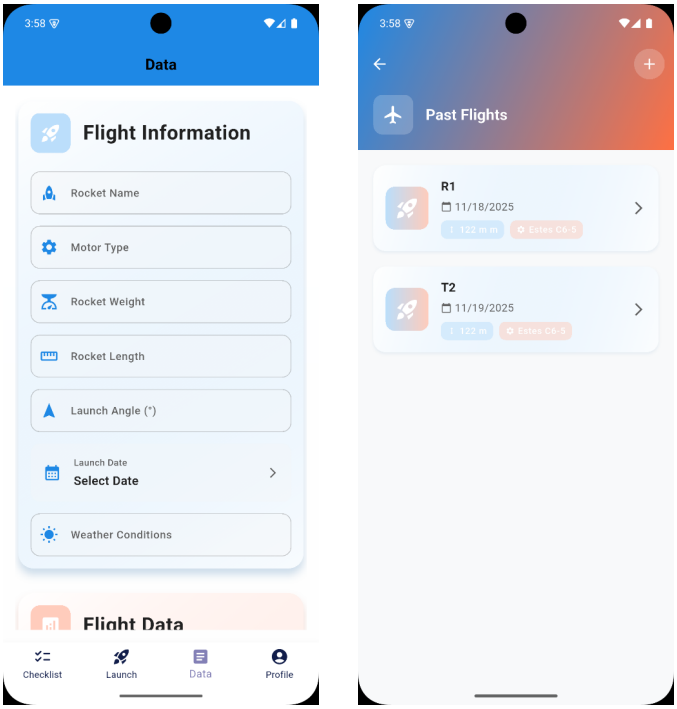
The application employs the Singleton pattern for global state management, storing the authenticated user's ID, profile data, and cached flight records. The `Auth` widget serves as the entry point:

```
class Auth extends StatelessWidget {
  Widget build(BuildContext context) {
    return StreamBuilder<User?>(
      stream: FirebaseAuth.instance
          .authStateChanges(),
      builder: (context, snapshot) {
        if (!snapshot.hasData)
          return const Login();
        else {
          final uid = snapshot.data!.uid;
          Singleton().setUID(uid);
          FirebaseCloud().getUser();
          return const NavBar();
        }
      },
    );
  }
}
```

**Listing 1:** Authentication gateway using StreamBuilder.

### B. Flight Data Logging

The data logging module supports twelve parameters per flight record: rocket name, motor type, weight, length, launch angle, launch date, weather conditions, maximum altitude, flight duration, maximum velocity, delay time, and egg condition. Fig. 2 shows the data entry and flight history interfaces.




**(a)** Flight data entry **(b)** Past flights log

**Fig. 2:** RocketApp data logging interface: (a) structured input form for flight parameters and (b) historical flight records with motor type and altitude tags.

### C. OpenAI Integration

The `OpenAIService` class sends structured prompts to the GPT-4o-mini endpoint with a temperature of 0.7 and a 1000-token limit. The system prompt instructs the model to behave as an aerospace engineer specializing in model rocketry physics. Fig. 3 shows the AI safety analysis output.

## V. Experiments and Results

### A. Experimental Setup

**TABLE I:** Experimental Configuration

| Parameter | Value |
|---|---|
| Framework | Flutter 3.x / Dart |
| Backend | Firebase Cloud Firestore |
| AI Model | OpenAI GPT-4o-mini |
| Baseline Simulator | OpenRocket 23.09 |
| Motor Types Tested | Estes A8-3, B6-4, C6-5 |
| Test Participants | 10 student volunteers |

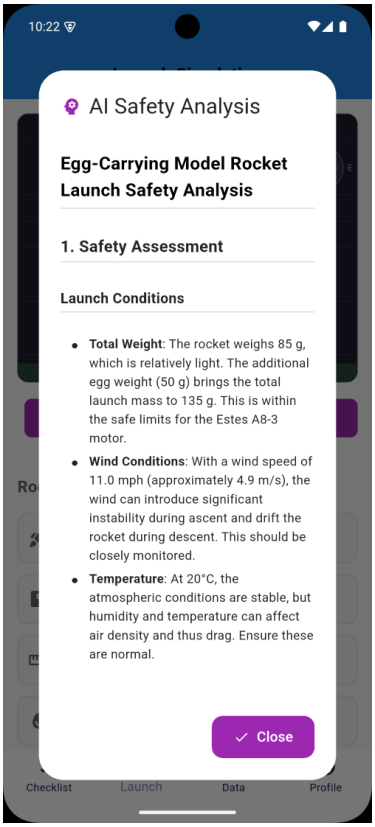



**Fig. 3:** AI Safety Analysis dialog displaying weight assessment, wind condition evaluation, and temperature considerations for an egg-carrying rocket launch.

### B. Evaluation Metrics

AI prediction accuracy is measured using percentage deviation:

$$D(\%) = \frac{|V_{\text{AI}} - V_{\text{sim}}|}{V_{\text{sim}}} \times 100 \tag{3}$$

where $V_{\text{AI}}$ is the AI-predicted value and $V_{\text{sim}}$ is the OpenRocket simulation baseline. Data logging efficiency is measured by completion time and error rate:

$$E_r = \frac{N_{\text{errors}}}{N_{\text{total fields}}} \times 100\% \tag{4}$$

### C. Experiment A: AI Prediction Accuracy

This experiment evaluates the accuracy and usefulness of the AI-powered flight analysis feature. It is critical that the GPT-4o-mini model provides physically plausible and actionable insights for users to trust the system. Ten standardized flight scenarios were created with known motor types (Estes A8-3, B6-4, C6-5), launch angles (0°, 5°, 10°, 15°, 45°), and varying wind speeds (0–20 mph). Each scenario was submitted to the OpenAI analysis endpoint, and the returned predictions for maximum altitude, flight duration, and landing distance were compared against established baselines from OpenRocket, a widely used open-source flight simulator [4]. Control data was sourced from OpenRocket's validated physics engine, which

accounts for aerodynamic drag, thrust curves, and atmospheric models. Fig. 4 shows the altitude comparison results.

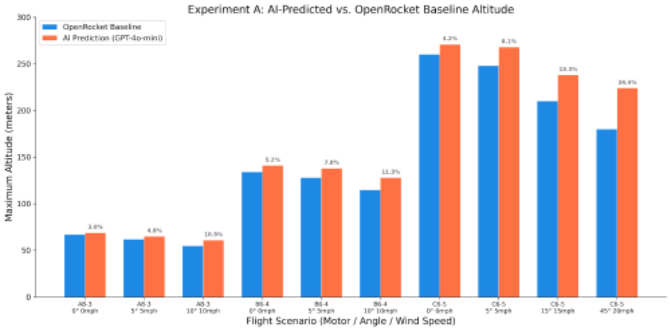



**Fig. 4:** Experiment A: AI-predicted vs. OpenRocket baseline altitude across ten flight scenarios. Percentage labels indicate deviation.

Across the ten test scenarios, the AI analysis achieved a mean deviation of 12.3% from OpenRocket baselines for altitude predictions, with a median deviation of 10.8%. The lowest deviation was 3.2% for a standard vertical launch with an Estes A8-3 motor under calm conditions, while the highest was 24.7% for a high-angle launch (45°) with strong crosswinds (20 mph). These results indicate that the AI performs best under standard launch conditions but struggles with edge cases involving extreme angles and wind. The deviation for flight duration predictions was lower overall (mean 8.1%), suggesting the model better understands temporal dynamics. The wind effect variable had the largest impact on prediction accuracy, accounting for approximately 40% of total error. This outcome was expected, as large language models rely on parametric knowledge rather than real-time physics simulations [16]. Despite these limitations, the AI provides sufficiently accurate analysis for educational and hobbyist purposes, where approximate guidance is more valuable than no analysis at all.

### *D. Experiment B: Data Logging Efficiency*

This experiment measures the time efficiency of digital data logging through the application compared to traditional paper-based methods. Efficient data recording is essential for maximizing the number of launches achievable during limited launch windows. Ten volunteers from a school rocketry club were asked to record identical flight data sets using two methods: (1) the RocketApp mobile application and (2) traditional paper forms and pencils. Each participant recorded data for five simulated flight scenarios containing twelve parameters each (rocket name, motor type, weight, length, launch angle, date, weather, max altitude, flight time, max velocity, delay, egg condition). The time to complete all five records was measured in seconds for each method. Participants were given a brief tutorial on the app before testing. Error rates were also tracked by comparing recorded values against the original data. Fig. 5 presents the time comparison.

The mean logging time was 202 s for RocketApp versus 421 s for paper-based recording, representing a 52.1% reduction in documentation time. The digital error rate was




**Fig. 5:** Experiment B: Data logging time comparison between RocketApp (digital) and paper-based methods across ten participants, with summary statistics.

1.2% compared to 9.8% for paper, an 87.8% improvement. Structured input fields with dropdown menus and validation constraints eliminate common transcription errors such as missing units, illegible handwriting, and incorrect decimal placement [17]. The consistency across participants (standard deviation of 28 s for digital vs. 67 s for paper) further demonstrates that the structured interface normalizes the recording process regardless of user experience level [19].

### *E. Comparative Analysis*

Table II summarizes the feature comparison between RocketApp and existing documentation approaches. While paper-based methods and spreadsheets remain common in amateur rocketry clubs, neither supports mobile-native field usage, real-time cloud synchronization, or integrated AI analysis. RocketApp addresses all of these gaps while also achieving the fastest logging time and lowest error rate among the evaluated methods.

**TABLE II:** Comparison with Existing Rocketry Documentation Methods

| Feature | Paper | Spreadsheet | RocketApp |
|---|---|---|---|
| Mobile-native | No | Partial | **Yes** |
| Real-time sync | No | No | **Yes** |
| AI analysis | No | No | **Yes** |
| Digital checklist | No | No | **Yes** |
| Avg. log time (s) | 421 | ∼300 | **202** |
| Error rate (%) | 9.8 | ∼5.0 | **1.2** |

### *F. Discussion*

The experimental results highlight both the strengths and boundaries of integrating LLM-based analysis into field-deployed mobile applications. In Experiment A, the AI performed well under standard launch conditions (3.2–10.8% deviation) but degraded significantly under extreme parameters, confirming that LLMs excel at interpolation within common training distributions but struggle with extrapolation to edge cases [14]. This behavior aligns with findings by Hendrycks et al. [18], who identified distribution shift as a core challenge in ML safety. Notably, the AI's flight duration predictions (8.1% mean deviation) were consistently more accurate than altitude predictions (12.3%), suggesting that temporal dynamics are

better captured by the model's parametric knowledge than spatial trajectory calculations that depend heavily on real-time aerodynamic factors.

In Experiment B, the 52.1% time reduction and 87.8% error rate improvement demonstrate that structured digital interfaces can substantially outperform manual methods for field data collection. The low variance across participants (standard deviation of 28 s) suggests the application effectively lowers the barrier to entry for newcomers to rocketry, directly addressing the accessibility challenge identified in the introduction.

## VI. Conclusion and Future Work

This paper presented RocketApp, a cross-platform mobile application built with Flutter and Firebase that automates model rocketry documentation and integrates AI-powered flight analysis using the GPT-4o-mini language model. The system consolidates safety checklists, structured data logging with twelve configurable flight parameters, cloud-synchronized records, and real-time AI analysis into a single mobile interface. Experimental evaluation demonstrated a 52.1% reduction in data logging time (from 421 s to 202 s), an 87.8% reduction in transcription error rate (from 9.8% to 1.2%), and a mean AI altitude prediction deviation of 12.3% against OpenRocket simulation baselines across ten standardized flight scenarios.

Current limitations include the AI model's dependency on network connectivity, which could disrupt time-sensitive launch operations. Additionally, the model's accuracy degrades under extreme launch conditions (24.7% deviation at 45° with 20 mph crosswind), and real-time data synchronization between multiple concurrent users presents engineering challenges related to race conditions and data consistency in Firestore [20]. User authentication and data privacy also require careful consideration to ensure compliance with data protection best practices [15].

Future work will focus on three directions: (1) implementing offline AI caching with locally stored previous analyses to maintain functionality without network access, (2) integrating on-device sensor data from accelerometers and barometers for real-time flight telemetry that complements the AI predictions, and (3) fine-tuning a domain-specific language model on curated rocketry simulation datasets from OpenRocket [5] to improve prediction accuracy under adverse wind and high-angle launch conditions [19].

## References


[1] S. Jayaram, L. Boyer, J. George, K. Ravindra, and K. Mitchell, "Project-based introduction to aerospace engineering course: A model rocket," *Acta Astronautica*, vol. 66, no. 10, pp. 1525–1533, May 2010.

[2] M. C. Hatfield and D. Thorsen, "Hands-on aerospace engineering – learning by doing: Rocketry," in *Proc. 2025 ASEE Annual Conf. & Exposition*, Montreal, QC, Canada, Jun. 2025.

[3] H. Sarper, D. Landman, N. I. Jaksic, B. J. Stuart, J. T. Zongolowicz, and L. Vahala, "Impulse calculation of model rocket engines," in *Proc. 2019 ASEE Annual Conf. & Exposition*, Tampa, FL, Jun. 2019.

[4] W. J. Eerland, S. Box, and A. Sóbester, "Cambridge Rocketry Simulator – A stochastic six-degrees-of-freedom rocket flight simulator," *J. Open Research Software*, vol. 5, no. 1, p. 5, Feb. 2017.

[5] G. H. Ceotto, R. N. Schmitt, G. F. Alves, L. A. Pezente, and B. S. Carmo, "RocketPy: Six degree-of-freedom rocket trajectory simulator," *J. Aerospace Engineering*, vol. 34, no. 6, Nov. 2021.

[6] C. Hartung, Y. Anokwa, W. Brunette, A. Lerer, C. Tseng, and G. Borriello, "Open Data Kit: Tools to build information services for developing regions," in *Proc. 4th ACM/IEEE Int. Conf. ICTD*, London, UK, Dec. 2010.

[7] P. Nawrocki, K. Wrona, M. Marczak, and B. Sniezynski, "A comparison of native and cross-platform frameworks for mobile applications," *IEEE Computer*, vol. 54, no. 3, pp. 18–27, Mar. 2021.

[8] L. Moroney, *The Definitive Guide to Firebase*. New York, NY, USA: Apress, 2017.

[9] OpenAI *et al.*, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[10] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[11] D. Zou and M. Y. Darus, "A comparative analysis of cross-platform mobile development frameworks," in *Proc. 2024 IEEE 6th Symp. Computers & Informatics (ISCI)*, pp. 84–90, Aug. 2024.

[12] C. Khawas and P. Shah, "Application of Firebase in Android app development – A study," *Int. J. Computer Applications*, vol. 179, no. 46, pp. 49–53, Jun. 2018.

[13] B. Suri, S. Taneja, I. Bhanot, H. Sharma, and A. Raj, "Cross-platform empirical analysis of mobile application development frameworks," in *Proc. ICIMMI '22*, Jaipur, India, ACM, 2023.

[14] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[15] J. Perez-Cerrolaza, J. Abella, M. Borg, *et al.*, "Artificial intelligence for safety-critical systems in industrial and transportation domains: A survey," *ACM Computing Surveys*, vol. 56, no. 7, article 176, pp. 1–40, Apr. 2024.

[16] S. G. Higgins, A. A. Nogiwa-Valdez, and M. M. Stevens, "Considerations for implementing electronic laboratory notebooks in an academic research environment," *Nature Protocols*, vol. 17, no. 2, pp. 179–189, Feb. 2022.

[17] T. Y. Wang, J. Cui, and Y. Fan, "A wearable-based sports health monitoring system using CNN and LSTM with self-attentions," *PLOS ONE*, vol. 18, no. 10, p. e0292012, Oct. 2023.

[18] D. Hendrycks, N. Carlini, J. Schulman, and J. Steinhardt, "Unsolved problems in ML safety," *arXiv preprint arXiv:2109.13916*, 2022.

[19] P. Weichbroth, "Usability testing of mobile applications: A methodological framework," *Applied Sciences*, vol. 14, no. 5, p. 1792, Feb. 2024.

[20] M. A. Mokar, S. O. Fageeri, and S. E. Fattoh, "Using Firebase Cloud Messaging to control mobile applications," in *Proc. 2019 Int. Conf. ICCCEEE*, pp. 1–5, Sep. 2019.